

**Міністерство освіти і науки України Національний технічний
університет України «Київський політехнічний інститут імені Ігоря
Сікорського» Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №4

з дисципліни

«Алгоритми і структури даних»

Виконала:

студентка групи ІМ-44

Крошка Дар'я Олександрівна

номер у списку групи: 11

Перевірив:

Сергієнко А. М.

Київ 2025

Завдання

Постановка задачі:

1. Представити напрямлений та ненапрямлений граф із заданими параметрами так само, як у лабораторній роботі №3. Відмінність: коефіцієнт k 1.0 n_3 0.01 n_4 0.01 0.3. Отже, матриця суміжності A_{dir} напрямленого графа за варіантом формується таким чином:
 - 1) встановлюється параметр (seed) генератора випадкових чисел, рівне номеру варіанту $n_1n_2n_3n_4$;
 - 2) матриця розміром $n \times n$ заповнюється згенерованими випадковими числами в діапазоні $[0, 2.0)$;
 - 3) обчислюється коефіцієнт k 1.0 n_3 0.01 n_4 0.01 0.3, кожен елемент матриці множиться на коефіцієнт k ;
 - 4) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 — якщо елемент більший або дорівнює 1.0.

Обчислити:

- 1) степені вершин напрямленого і ненапрямленого графів;
 - 2) напівстепені виходу та заходу напрямленого графа;
 - 3) чи є граф однорідним (регулярним), і якщо так, вказати степінь однорідності графа;
 - 4) перелік висячих та ізольованих вершин. Результати вивести у графічне вікно, консоль або файл.
3. Змінити матрицю A_{dir} , коефіцієнт k 1.0 n_3 0.005 n_4 0.005 0.27.
 4. Для нового орграфа обчислити:
 - 1) півстепені вершин;
 - 2) всі шляхи довжини 2 і 3;
 - 3) матрицю досяжності;
 - 4) матрицю сильної зв'язності;
 - 5) перелік компонент сильної зв'язності;
 - 6) граф конденсації. Результати вивести у графічне вікно, в консоль або файл. Шляхи довжиною 2 і 3 слід шукати за матрицями A_2 і A_3 , відповідно. Як результат вивести перелік шляхів, включно з усіма проміжними вершинами, через які проходить шлях. Матрицю досяжності та компоненти сильної зв'язності слід шукати за допомогою операції транзитивного замикання. У переліку компонент слід вказати, які вершини належать до кожної компоненти. Граф конденсації вивести у графічне вікно.

Текст програми:

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Лабораторна робота 4</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    }

    body {
      background: linear-gradient(135deg, #E3F2FD, #BBDEFB);
      min-height: 100vh;
      padding: 20px;
    }

    .container {
      max-width: 1400px;
      margin: 0 auto;
    }

    h1 {
      color: #0D47A1;
      font-size: 2.4rem;
      font-weight: 700;
      margin-bottom: 30px;
      text-align: center;
      text-shadow: 1px 1px 6px rgba(0, 0, 0, 0.1);
    }

    .section {
      background: rgba(255, 255, 255, 0.95);
```

```
border-radius: 16px;
padding: 25px;
margin: 20px 0;
box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1);
border: 2px solid #BBDEFB;
}
```

```
.section h2 {
  color: #1565C0;
  font-size: 1.5rem;
  margin-bottom: 20px;
  border-bottom: 2px solid #BBDEFB;
  padding-bottom: 10px;
}
```

```
.canvases {
  display: flex;
  flex-wrap: wrap;
  gap: 30px;
  justify-content: center;
  margin: 20px 0;
}
```

```
canvas {
  background-color: #FFFFFF;
  border: 4px solid #1565C0;
  border-radius: 16px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}
```

```
canvas:hover {
  transform: scale(1.02);
  box-shadow: 0 12px 36px rgba(0, 0, 0, 0.2);
}
```

```
.results {
```

```
display: grid;
grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
gap: 20px;
}
```

```
.result-box {
  background: #F8F9FA;
  border: 2px solid #E3F2FD;
  border-radius: 12px;
  padding: 15px;
  font-family: 'Courier New', monospace;
  font-size: 14px;
  line-height: 1.4;
  color: #0D47A1;
  max-height: 300px;
  overflow-y: auto;
}
```

```
.result-box h3 {
  color: #1565C0;
  font-size: 16px;
  margin-bottom: 10px;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
```

```
.highlight {
  background-color: #E3F2FD;
  padding: 2px 4px;
  border-radius: 4px;
  font-weight: bold;
}
```

```
.matrix {
  font-family: 'Courier New', monospace;
  font-size: 12px;
  line-height: 1.2;
  white-space: pre;
```

```

    }

    @media (max-width: 768px) {
        canvas {
            width: 90vw;
            height: 90vw;
            max-width: 400px;
            max-height: 400px;
        }

        .results {
            grid-template-columns: 1fr;
        }
    }
</style>
</head>
<body>
    <div class="container">
        <h1>Лабораторна робота 4</h1>

        <div class="section">
            <h2>Завдання 1: Графи з першим коефіцієнтом  $k_1$ </h2>
            <div class="canvases">
                <canvas id="directedGraphCanvas1" width="500" height="500"
title="Напрямлений граф ( $k_1$ )"></canvas>
                <canvas id="undirectedGraphCanvas1" width="500" height="500"
title="Ненапрямлений граф ( $k_1$ )"></canvas>
            </div>
        </div>

        <div class="section">
            <h2>Завдання 2: Аналіз графів ( $k_1$ )</h2>
            <div class="results" id="task2Results"></div>
        </div>

        <div class="section">

```

<h2>Завдання 3-4: Графи з другим коефіцієнтом k_2 та повний аналіз</h2>

```
<div class="canvases">
  <canvas id="directedGraphCanvas2" width="500" height="500"
title="Напрямлений граф ( $k_2$ )"></canvas>
  <canvas id="condensedGraphCanvas" width="500" height="500"
title="Граф конденсації"></canvas>
</div>
</div>
```

```
<div class="section">
  <h2>Завдання 4: Результати аналізу оргграфа ( $k_2$ )</h2>
  <div class="results" id="task4Results"></div>
</div>
</div>
```

```
<script
src="https://cdnjs.cloudflare.com/ajax/libs/seedrandom/3.0.5/seedrandom.min.js
"></script>
```

```
<script>
  const n = 11;
  const radius = 15;
  const centerX = 250, centerY = 250, circleRadius = 180;
  const seed = 4411;
  const n3 = 1, n4 = 1;

  const k1 = 1.0 - n3 * 0.01 - n4 * 0.01 - 0.3;
  const k2 = 1.0 - n3 * 0.005 - n4 * 0.005 - 0.27;

  console.log(`k1 = ${k1}, k2 = ${k2}`);

  const nodePositions = Array.from({ length: n }, (_, i) => {
    const angle = (i * 2 * Math.PI) / n;
    return {
      x: centerX + circleRadius * Math.cos(angle),
      y: centerY + circleRadius * Math.sin(angle)
    };
  });
```

```
});
```

```
function generateAdjacencyMatrix(k, directed = true) {  
  let matrix = [];  
  let rng = new Math.seedrandom(seed);  
  for (let i = 0; i < n; i++) {  
    matrix[i] = [];  
    for (let j = 0; j < n; j++) {  
      let value = rng() * 2.0 * k;  
      matrix[i][j] = value >= 1.0 ? 1 : 0;  
    }  
  }  
  if (!directed) {  
    for (let i = 0; i < n; i++) {  
      for (let j = i + 1; j < n; j++) {  
        if (matrix[i][j] === 1) matrix[j][i] = 1;  
      }  
    }  
  }  
  return matrix;  
}
```

```
function drawGraph(ctx, matrix, directed, title) {  
  ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);  
  
  ctx.fillStyle = "#1565C0";  
  ctx.font = "16px Arial";  
  ctx.textAlign = "center";  
  ctx.fillText(title, centerX, 30);  
  
  drawEdges(ctx, matrix, directed);  
  drawNodes(ctx, nodePositions);  
}
```

```
function drawNodes(ctx, positions) {  
  positions.forEach((pos, idx) => {  
    ctx.fillStyle = "#1565C0";
```



```

    ctx.beginPath();
    ctx.arc(pos.x, pos.y, radius, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "#0D47A1";
    ctx.lineWidth = 2;
    ctx.stroke();

    ctx.fillStyle = "white";
    ctx.font = "12px Arial";
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";
    ctx.fillText(idx + 1, pos.x, pos.y);
  });
}

function drawEdges(ctx, matrix, directed) {
  ctx.strokeStyle = "#0D47A1";
  ctx.lineWidth = 1.5;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if (matrix[i][j] === 1) {
        const { x: x1, y: y1 } = nodePositions[i];
        const { x: x2, y: y2 } = nodePositions[j];
        const angle = Math.atan2(y2 - y1, x2 - x1);
        const x1_new = x1 + radius * Math.cos(angle);
        const y1_new = y1 + radius * Math.sin(angle);
        const x2_new = x2 - radius * Math.cos(angle);
        const y2_new = y2 - radius * Math.sin(angle);

        ctx.beginPath();
        ctx.moveTo(x1_new, y1_new);
        ctx.lineTo(x2_new, y2_new);
        ctx.stroke();

        if (directed) {
          drawArrow(ctx, x2_new, y2_new, angle);
        }
      }
    }
  }
}

```

```

    }
  }
}

```

```

function drawArrow(ctx, x, y, angle) {
  const size = 8;
  ctx.save();
  ctx.translate(x, y);
  ctx.rotate(angle);
  ctx.fillStyle = "#0D47A1";
  ctx.beginPath();
  ctx.moveTo(0, 0);
  ctx.lineTo(-size, -size / 2);
  ctx.lineTo(-size, size / 2);
  ctx.closePath();
  ctx.fill();
  ctx.restore();
}

```

```

function calculateDegrees(matrix, directed) {
  const degrees = [];
  const inDegrees = [];
  const outDegrees = [];

  for (let i = 0; i < n; i++) {
    let inDeg = 0, outDeg = 0;
    for (let j = 0; j < n; j++) {
      if (matrix[i][j] === 1) outDeg++;
      if (matrix[j][i] === 1) inDeg++;
    }

    if (directed) {
      inDegrees.push(inDeg);
      outDegrees.push(outDeg);
      degrees.push(inDeg + outDeg);
    } else {

```

```

        degrees.push(inDeg);
    }
}

return directed ? { degrees, inDegrees, outDegrees } : { degrees };
}

function checkRegularity(degrees) {
    const firstDegree = degrees[0];
    const isRegular = degrees.every(deg => deg === firstDegree);
    return { isRegular, degree: isRegular ? firstDegree : null };
}

function findSpecialVertices(degrees) {
    const pendant = [];
    const isolated = [];

    degrees.forEach((deg, idx) => {
        if (deg === 0) isolated.push(idx + 1);
        else if (deg === 1) pendant.push(idx + 1);
    });

    return { pendant, isolated };
}

function matrixMultiply(A, B) {
    const result = Array.from({ length: n }, () => Array(n).fill(0));
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            for (let k = 0; k < n; k++) {
                result[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    return result;
}

```

```

function findPathsFromMatrix(matrix, length) {
  let powerMatrix = matrix;
  for (let i = 1; i < length; i++) {
    powerMatrix = matrixMultiply(powerMatrix, matrix);
  }

  const paths = [];
  const pathDetails = [];

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if (powerMatrix[i][j] > 0) {
        const actualPaths = findAllPathsOfLength(matrix, i, j, length);
        actualPaths.forEach(path => {
          const pathStr = path.map(v => v + 1).join(' → ');
          paths.push(pathStr);
          pathDetails.push({
            from: i + 1,
            to: j + 1,
            path: path.map(v => v + 1),
            pathString: pathStr
          });
        });
      }
    }
  }

  return { paths, pathDetails, matrix: powerMatrix };
}

function findAllPathsOfLength(matrix, start, end, length) {
  if (length === 1) {
    return matrix[start][end] === 1 ? [[start, end]] : [];
  }

  const paths = [];

```

```

function dfs(current, target, currentLength, currentPath) {
  if (currentLength === length) {
    if (current === target) {
      paths.push([...currentPath]);
    }
    return;
  }

  for (let next = 0; next < n; next++) {
    if (matrix[current][next] === 1) {
      currentPath.push(next);
      dfs(next, target, currentLength + 1, currentPath);
      currentPath.pop();
    }
  }
}

dfs(start, end, 0, [start]);
return paths;
}

function calculateReachabilityMatrix(matrix) {
  let reach = matrix.map(row => [...row]);

  for (let k = 0; k < n; k++) {
    for (let i = 0; i < n; i++) {
      for (let j = 0; j < n; j++) {
        reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);
      }
    }
  }

  return reach.map(row => row.map(val => val ? 1 : 0));
}

function calculateStrongConnectivityMatrix(matrix) {
  const reach = calculateReachabilityMatrix(matrix);

```

```

const transpose = transposeMatrix(matrix);
const reachTranspose = calculateReachabilityMatrix(transpose);

const strongMatrix = Array.from({ length: n }, () => Array(n).fill(0));

for (let i = 0; i < n; i++) {
  for (let j = 0; j < n; j++) {
    strongMatrix[i][j] = (reach[i][j] && reachTranspose[j][i]) ? 1 : 0;
  }
}

return strongMatrix;
}

function transposeMatrix(matrix) {
  return matrix[0].map((_, colIndex) => matrix.map(row =>
row[colIndex]));
}

function kosaraju(matrix) {
  let visited = new Array(n).fill(false);
  let stack = [];
  let components = [];

  function dfs1(v) {
    visited[v] = true;
    for (let i = 0; i < n; i++) {
      if (matrix[v][i] === 1 && !visited[i]) {
        dfs1(i);
      }
    }
    stack.push(v);
  }

  function dfs2(v, component, transposed) {
    visited[v] = true;
    component.push(v);
  }

```

```

    for (let i = 0; i < n; i++) {
        if (transposed[v][i] === 1 && !visited[i]) {
            dfs2(i, component, transposed);
        }
    }
}

for (let i = 0; i < n; i++) {
    if (!visited[i]) dfs1(i);
}

let transposed = transposeMatrix(matrix);
visited.fill(false);

while (stack.length > 0) {
    let v = stack.pop();
    if (!visited[v]) {
        let component = [];
        dfs2(v, component, transposed);
        components.push(component);
    }
}
return components;
}

function buildCondensationGraph(components, componentMap, matrix) {
    const size = components.length;
    const condensed = Array.from({ length: size }, () => Array(size).fill(0));
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            if (matrix[i][j]) {
                const ci = componentMap[i];
                const cj = componentMap[j];
                if (ci !== cj) {
                    condensed[ci][cj] = 1;
                }
            }
        }
    }
}

```

```

    }
  }
  return condensed;
}

```

```

function drawCondensedGraph(ctx, condensedMatrix, components) {
  const size = condensedMatrix.length;
  const positions = Array.from({ length: size }, (_, i) => {
    const angle = (i * 2 * Math.PI) / size;
    return {
      x: centerX + circleRadius * Math.cos(angle),
      y: centerY + circleRadius * Math.sin(angle)
    };
  });
});

```

```

ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);

```

```

ctx.fillStyle = "#1565C0";
ctx.font = "16px Arial";
ctx.textAlign = "center";
ctx.fillText("Граф конденсації", centerX, 30);

```

```

ctx.strokeStyle = "#0D47A1";
ctx.lineWidth = 2;
condensedMatrix.forEach((row, i) => {
  row.forEach((val, j) => {
    if (val === 1) {
      const { x: x1, y: y1 } = positions[i];
      const { x: x2, y: y2 } = positions[j];
      const angle = Math.atan2(y2 - y1, x2 - x1);
      const x1_new = x1 + radius * Math.cos(angle);
      const y1_new = y1 + radius * Math.sin(angle);
      const x2_new = x2 - radius * Math.cos(angle);
      const y2_new = y2 - radius * Math.sin(angle);
      ctx.beginPath();
      ctx.moveTo(x1_new, y1_new);
      ctx.lineTo(x2_new, y2_new);
    }
  });
});

```



```

        ctx.stroke();
        drawArrow(ctx, x2_new, y2_new, angle);
    }
});
});

```

```

positions.forEach((pos, idx) => {
    ctx.fillStyle = "#1565C0";
    ctx.beginPath();
    ctx.arc(pos.x, pos.y, radius + 5, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "#0D47A1";
    ctx.lineWidth = 3;
    ctx.stroke();

```

```

    ctx.fillStyle = "white";
    ctx.font = "12px Arial";
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";
    ctx.fillText(`C${idx + 1}`, pos.x, pos.y);

```

```

    const componentText = `${components[idx].map(v => v +
1).join(',')}`;
    ctx.fillStyle = "#0D47A1";
    ctx.font = "10px Arial";
    ctx.fillText(componentText, pos.x, pos.y + radius + 15);
});
}

```

```

function formatMatrix(matrix, title) {
    let result = `${title}:\n`;
    result += ' ';
    for (let j = 0; j < matrix[0].length; j++) {
        result += String(j + 1).padStart(3);
    }
    result += '\n';

```

```

    for (let i = 0; i < matrix.length; i++) {
      result += String(i + 1).padStart(2) + ' ';
      for (let j = 0; j < matrix[i].length; j++) {
        result += String(matrix[i][j]).padStart(3);
      }
      result += '\n';
    }
    return result;
  }
}

```

```

function displayResults(containerId, results) {
  const container = document.getElementById(containerId);
  container.innerHTML = "";

  results.forEach(result => {
    const box = document.createElement('div');
    box.className = 'result-box';

    const title = document.createElement('h3');
    title.textContent = result.title;
    box.appendChild(title);

    const content = document.createElement('div');
    if (result.isMatrix) {
      content.className = 'matrix';
    }
    content.innerHTML = result.content;
    box.appendChild(content);

    container.appendChild(box);
  });
}

```

```

function runAnalysis() {
  const directedMatrix1 = generateAdjacencyMatrix(k1, true);
  const undirectedMatrix1 = generateAdjacencyMatrix(k1, false);
  const directedMatrix2 = generateAdjacencyMatrix(k2, true);

```

```

const directedCtx1 =
document.getElementById('directedGraphCanvas1').getContext('2d');
const undirectedCtx1 =
document.getElementById('undirectedGraphCanvas1').getContext('2d');
const directedCtx2 =
document.getElementById('directedGraphCanvas2').getContext('2d');
const condensedCtx =
document.getElementById('condensedGraphCanvas').getContext('2d');

drawGraph(directedCtx1, directedMatrix1, true, `Направлений граф
(k1=${k1.toFixed(3)})`);
drawGraph(undirectedCtx1, undirectedMatrix1, false, `Ненаправлений
граф (k1=${k1.toFixed(3)})`);
drawGraph(directedCtx2, directedMatrix2, true, `Направлений граф
(k2=${k2.toFixed(3)})`);

const directedDegrees1 = calculateDegrees(directedMatrix1, true);
const undirectedDegrees1 = calculateDegrees(undirectedMatrix1, false);
const directedRegularity = checkRegularity(directedDegrees1.degrees);
const undirectedRegularity =
checkRegularity(undirectedDegrees1.degrees);
const directedSpecial = findSpecialVertices(directedDegrees1.degrees);
const undirectedSpecial =
findSpecialVertices(undirectedDegrees1.degrees);

const task2Results = [
  {
    title: 'Степені вершин напрямленого графа',
    content: directedDegrees1.degrees.map((deg, i) => `Вершина
${i+1}: ${deg}`).join('<br>')
  },
  {
    title: 'Напівстепені заходу',
    content: directedDegrees1.inDegrees.map((deg, i) => `Вершина
${i+1}: ${deg}`).join('<br>')
  },

```

```

    {
      title: 'Напівстепені виходу',
      content: directedDegrees1.outDegrees.map((deg, i) => `Вершина
${i+1}: ${deg}`).join('<br>')
    },
    {
      title: 'Степені вершин ненапрямленого графа',
      content: undirectedDegrees1.degrees.map((deg, i) => `Вершина
${i+1}: ${deg}`).join('<br>')
    },
    {
      title: 'Регулярність напрямленого графа',
      content: directedRegularity.isRegular
        ? `Граф регулярний, степінь:
${directedRegularity.degree}</span>`
        : 'Граф нерегулярний'
    },
    {
      title: 'Регулярність ненапрямленого графа',
      content: undirectedRegularity.isRegular
        ? `Граф регулярний, степінь:
${undirectedRegularity.degree}</span>`
        : 'Граф нерегулярний'
    },
    {
      title: 'Висячі та ізольовані вершини (напрямлений)',
      content: `Висячі: ${directedSpecial.pendant.length ?
directedSpecial.pendant.join(', ') : 'немає'}<br>
        Ізольовані: ${directedSpecial.isolated.length ?
directedSpecial.isolated.join(', ') : 'немає'}`
    },
    {
      title: 'Висячі та ізольовані вершини (ненапрямлений)',
      content: `Висячі: ${undirectedSpecial.pendant.length ?
undirectedSpecial.pendant.join(', ') : 'немає'}<br>
        Ізольовані: ${undirectedSpecial.isolated.length ?
undirectedSpecial.isolated.join(', ') : 'немає'}`
    }
  ]

```

```

    }
  ];

  displayResults('task2Results', task2Results);

  const directedDegrees2 = calculateDegrees(directedMatrix2, true);
  const paths2Result = findPathsFromMatrix(directedMatrix2, 2);
  const paths3Result = findPathsFromMatrix(directedMatrix2, 3);
  const reachabilityMatrix =
calculateReachabilityMatrix(directedMatrix2);
  const strongConnectivityMatrix =
calculateStrongConnectivityMatrix(directedMatrix2);
  const components = kosaraju(directedMatrix2);

  const componentMap = Array(n).fill(0);
  components.forEach((component, idx) => {
    component.forEach(node => {
      componentMap[node] = idx;
    });
  });

  const condensedMatrix = buildCondensationGraph(components,
componentMap, directedMatrix2);
  drawCondensedGraph(condensedCtx, condensedMatrix, components);

  const task4Results = [
    {
      title: 'Напівстепені вершин ( $k_2$ )',
      content: `Напівстепені заходу:
 $\{ \text{directedDegrees2.inDegrees.map}((\text{deg}, i) => \text{\`}\{i+1\}:\{\text{deg}\}\text{\`}).join(', ')\}$ <br>
Напівстепені виходу:
 $\{ \text{directedDegrees2.outDegrees.map}((\text{deg}, i) => \text{\`}\{i+1\}:\{\text{deg}\}\text{\`}).join(', ')\}$ 
`,
    },
    {
      title: 'Матриця  $A^2$  (шляхи довжини 2)',
      content: formatMatrix(paths2Result.matrix, 'A2'),
      isMatrix: true
    }
  ]

```

```

    },
    {
      title: 'Шляхи довжини 2',
      content: paths2Result.paths.length ? paths2Result.paths.slice(0,
20).join('<br>') +
        (paths2Result.paths.length > 20 ? '<br>... та інші' : '') +
        `<br><br><span class="highlight">Всього шляхів:
${paths2Result.paths.length}</span>` : 'Немає шляхів'
    },
    {
      title: 'Матриця  $A^3$  (шляхи довжини 3)',
      content: formatMatrix(paths3Result.matrix, 'A3'),
      isMatrix: true
    },
    {
      title: 'Шляхи довжини 3',
      content: paths3Result.paths.length ? paths3Result.paths.slice(0,
15).join('<br>') +
        (paths3Result.paths.length > 15 ? '<br>... та інші' : '') +
        `<br><br><span class="highlight">Всього шляхів:
${paths3Result.paths.length}</span>` : 'Немає шляхів'
    },
    {
      title: 'Матриця досяжності',
      content: formatMatrix(reachabilityMatrix, 'R'),
      isMatrix: true
    },
    {
      title: 'Матриця сильної зв'язності',
      content: formatMatrix(strongConnectivityMatrix, 'S'),
      isMatrix: true
    },
    {
      title: 'Компоненти сильної зв'язності',
      content: components.map((comp, i) =>
        `Компонента ${i+1}: ${comp.map(v => v+1).join(', ')}`
      )
    }
  ]

```

```

    ).join('<br>') + `<br><br><span class="highlight">Всього
компонент: ${components.length}</span>`
  }
];

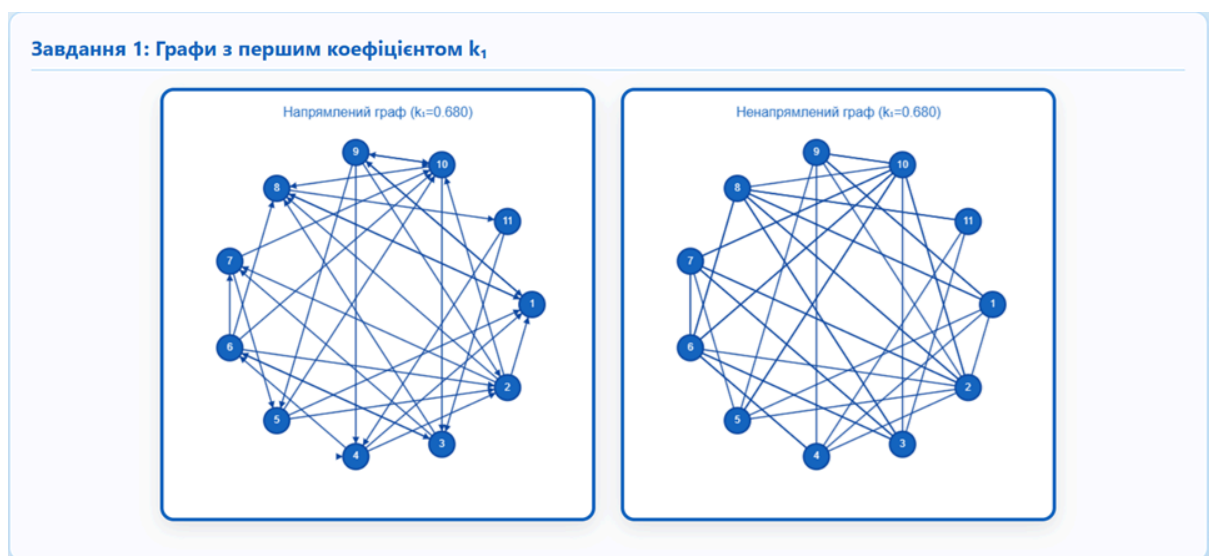
displayResults('task4Results', task4Results);

console.log('=== РЕЗУЛЬТАТИ АНАЛІЗУ ===');
console.log(`k1 = ${k1}, k2 = ${k2}`);
console.log('Компоненти сильної зв'язності:', components.map(c =>
с.map(i => i + 1)));
console.log(`Кількість компонент: ${components.length}`);
console.log('Шляхи довжини 2:', paths2Result.paths);
console.log('Шляхи довжини 3:', paths3Result.paths);
}

window.addEventListener('load', runAnalysis);
</script>
</body>
</html>

```

Скріншоти роботи програми



Завдання 2: Аналіз графів (k_1)

Степені вершин напрямленого графа

Вершина 1: 7
Вершина 2: 8
Вершина 3: 6
Вершина 4: 7
Вершина 5: 5
Вершина 6: 7
Вершина 7: 5
Вершина 8: 7
Вершина 9: 7
Вершина 10: 10
Вершина 11: 3

Напівстепені заходу

Вершина 1: 5
Вершина 2: 4
Вершина 3: 3
Вершина 4: 3
Вершина 5: 2
Вершина 6: 2
Вершина 7: 3
Вершина 8: 5
Вершина 9: 2
Вершина 10: 6
Вершина 11: 1

Напівстепені виходу

Вершина 1: 2
Вершина 2: 4
Вершина 3: 3
Вершина 4: 4
Вершина 5: 3
Вершина 6: 5
Вершина 7: 2
Вершина 8: 2
Вершина 9: 5
Вершина 10: 4
Вершина 11: 2

Степені вершин ненапрямленого графа

Вершина 1: 5
Вершина 2: 7
Вершина 3: 5
Вершина 4: 4
Вершина 5: 3
Вершина 6: 5
Вершина 7: 4
Вершина 8: 6
Вершина 9: 2
Вершина 10: 6
Вершина 11: 1

Регулярність напрямленого графа

Граф нерегулярний

Регулярність ненапрямленого графа

Граф нерегулярний

Висячі та ізольовані вершини (напрямлений)

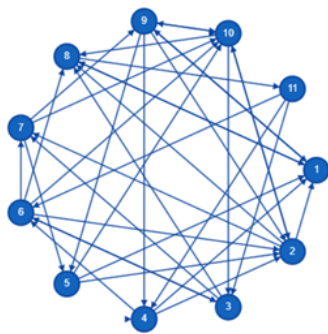
Висячі: немає
Ізольовані: немає

Висячі та ізольовані вершини (ненапрямлений)

Висячі: 11
Ізольовані: немає

Завдання 3-4: Графи з другим коефіцієнтом k_2 та повний аналіз

Напрямлений граф ($k_2=0.720$)



Граф конденсації

C1
(1,2,4,6,7,3,5,11,8,10,9)

Завдання 4: Результати аналізу оргграфа (k_2)

Напівстепені вершин (k_2)

Напівстепені заходу: 1:5, 2:5, 3:3, 4:3, 5:2, 6:3, 7:3, 8:5, 9:3, 10:6, 11:1
Напівстепені виходу: 1:2, 2:4, 3:3, 4:4, 5:3, 6:5, 7:3, 8:2, 9:5, 10:5, 11:3

Матриця A^2 (шляхи довжини 2)

A^2 :

1	1	2	3	4	5	6	7	8	9	10	11
1	2	1	0	1	1	0	0	0	0	1	1
2	1	1	1	0	1	0	0	2	3	2	1
3	1	1	1	0	1	0	1	1	1	1	2
4	2	2	1	1	0	1	2	3	1	2	0
5	1	1	1	0	0	0	1	3	2	2	0
6	2	1	1	0	1	1	2	3	2	3	1
7	2	3	1	1	1	0	0	1	1	3	0
8	0	0	1	1	0	1	0	1	1	0	0
9	3	3	1	1	0	1	1	3	2	3	0
10	3	2	1	1	1	1	2	3	1	3	1
11	1	2	1	1	0	2	2	2	0	1	0

Шляхи довжини 2

1 → 8 → 1
1 → 9 → 1
1 → 9 → 2
1 → 9 → 4
1 → 9 → 5
1 → 9 → 10
1 → 8 → 11
2 → 8 → 1
2 → 10 → 2
2 → 10 → 3
2 → 7 → 5
2 → 1 → 8
2 → 10 → 8

Матриця A^3 (шляхи довжини 3)

A^3 :

1	1	2	3	4	5	6	7	8	9	10	11
1	3	3	2	2	0	2	1	4	3	3	0
2	7	6	3	4	3	2	2	5	3	7	2
3	4	4	3	2	2	2	2	5	4	6	1
4	7	5	3	2	3	2	4	8	6	8	3
5	6	4	2	2	3	1	2	5	4	6	3
6	7	7	5	3	4	2	3	8	7	10	3
7	6	3	2	1	2	4	9	5	8	1	
8	3	3	1	2	1	2	2	2	0	2	1
9	9	7	4	3	3	2	5	11	7	10	3
10	8	7	5	3	3	3	4	10	8	10	3
11	5	4	3	1	2	2	5	7	4	7	2

Шляхи довжини 3

1 → 9 → 2 → 1
1 → 9 → 4 → 1
1 → 9 → 5 → 1
1 → 9 → 4 → 2
1 → 9 → 5 → 2
1 → 9 → 10 → 2
1 → 8 → 11 → 3
1 → 9 → 10 → 3
1 → 8 → 11 → 4
1 → 9 → 4 → 4
1 → 8 → 11 → 6
1 → 9 → 4 → 6
1 → 9 → 2 → 7

Матриця досяжності

R :

1	1	2	3	4	5	6	7	8	9	10	11
1	1	1	1	1	1	1	1	1	1	1	1
2	1	1	1	1	1	1	1	1	1	1	1
3	1	1	1	1	1	1	1	1	1	1	1
4	1	1	1	1	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1	1	1	1	1
6	1	1	1	1	1	1	1	1	1	1	1
7	1	1	1	1	1	1	1	1	1	1	1
8	1	1	1	1	1	1	1	1	1	1	1
9	1	1	1	1	1	1	1	1	1	1	1
10	1	1	1	1	1	1	1	1	1	1	1
11	1	1	1	1	1	1	1	1	1	1	1

Матриця сильної зв'язності

S :

1	1	2	3	4	5	6	7	8	9	10	11
1	1	1	1	1	1	1	1	1	1	1	1
2	1	1	1	1	1	1	1	1	1	1	1
3	1	1	1	1	1	1	1	1	1	1	1
4	1	1	1	1	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1	1	1	1	1
6	1	1	1	1	1	1	1	1	1	1	1
7	1	1	1	1	1	1	1	1	1	1	1
8	1	1	1	1	1	1	1	1	1	1	1
9	1	1	1	1	1	1	1	1	1	1	1
10	1	1	1	1	1	1	1	1	1	1	1
11	1	1	1	1	1	1	1	1	1	1	1

Компоненти сильної зв'язності

Компонента 1: {1, 2, 4, 9, 7, 3, 6, 11, 8, 10, 5}

Всього компонент: 1


```

k1 = 0.6799999999999999, k2 = 0.72
=== РЕЗУЛЬТАТИ АНАЛІЗУ ===
k1 = 0.6799999999999999, k2 = 0.72
Компоненти сильної зв'язності: ▼ Array(1) ⓘ
  ▶ 0: (11) [1, 2, 4, 9, 7, 3, 6, 11, 8, 10, 5]
    length: 1
  ▶ [[Prototype]]: Array(0)

Кількість компонент: 1

Шляхи довжини 2: ▼ Array(139) ⓘ
  ▶ [0 ... 99]
  ▶ [100 ... 138]
    length: 139
  ▶ [[Prototype]]: Array(0)

Шляхи довжини 3: ▼ Array(487) ⓘ
  ▶ [0 ... 99]
  ▶ [100 ... 199]
  ▶ [200 ... 299]
  ▶ [300 ... 399]
  ▶ [400 ... 486]
    length: 487
  ▶ [[Prototype]]: Array(0)

```

Висновки

У рамках виконання лабораторної роботи були розглянуті напрямлені та ненаправлені графи, побудовані на основі випадкових значень, згенерованих за допомогою визначеного коефіцієнта kkk . Програма успішно реалізувала побудову матриць суміжності для обох типів графів, а також визначила важливі характеристики графів. Результати були візуалізовані на графах, що дозволило детально вивчити структуру зв'язків у графах. Програма працює коректно і надає точні результати для всіх вказаних характеристик графів. Загалом, лабораторна робота допомогла глибше зрозуміти методи аналізу графів та їх властивості, використовуючи алгоритми для пошуку компонент сильної зв'язності та матриць досяжності.